

Assignment: DFS and BFS Implementation + Reuse

Objective: Implement classical DFS and BFS once (Task 1) and then solve multiple problems (Task 2) by reusing those implementations with only minimal modifications (neighbor generation, conditions, state representation). Rewriting DFS/BFS from scratch per problem is not allowed.

Task 1 - Core implementations (required)

- 1) Implement DFS as a reusable function (recursive or iterative stack).
- 2) Implement BFS as a reusable function using a queue.
- 3) In main(), demonstrate both traversals on a small example to prove they work.

Reuse rule for Task 2: You must call your Task 1 DFS/BFS functions. You may adjust only the parts that define neighbors, visited checks, stop conditions, and the state stored in recursion/queue.

Task 2 - Solve problems using your Task 1 DFS/BFS

For every problem below, include a short demonstration in main() and print results clearly. Each table provides three example test cases: at least two are solvable and exactly one demonstrates a 'no solution' / negative outcome.

DFS problems

D1. Maze path existence (DFS on grid)

Given a 2D grid of 0/1 (0=open, 1=blocked), start at (0,0) and determine whether you can reach (n-1,m-1) using 4-direction moves.

Example input	Example output
grid = [[0,0,1],[1,0,1],[1,0,0]]	YES
grid = [[0,0],[0,0]]	YES
grid = [[0,1],[1,0]]	NO

D2. Restore IP Addresses (DFS/backtracking)

Given a string of digits s, return all valid IP addresses by inserting three dots (4 parts, each 0-255; no leading zeros unless the part is exactly '0').

Example input	Example output
s = "25525511135"	["255.255.11.135", "255.255.111.35"]
s = "101023"	["1.0.10.23", "1.0.102.3", "10.1.0.23", "10.10.2.3", "101.0.2.3"]
s = "9999999999999"	[]

D3. Combination Sum (DFS/backtracking)

Given an array of positive integers candidates and a target T, find all combinations that sum to T. (Variant: candidates may be reused.)

Example input	Example output
candidates = [2,3,6,7], T = 7	[[2,2,3], [7]]
candidates = [2,3,5], T = 8	[[2,2,2,2], [2,3,3], [3,5]]
candidates = [4,6], T = 5	[]

D4. Generate balanced parentheses (DFS)

Input n. Output all strings with n pairs of parentheses that are properly balanced.

Example input	Example output
n = 1	["()"]
n = 3	["((()))", "(()())", "(()())", "()(())", "()()()"]
n = -1	[] (invalid n)

D5. Cycle detection in an undirected graph (DFS)

Given an undirected graph, determine whether it contains a cycle.

Example input	Example output
n=3, edges=[(0,1),(1,2),(2,0)]	YES
n=4, edges=[(0,1),(1,2),(2,3),(3,0)]	YES
n=4, edges=[(0,1),(1,2),(2,3)]	NO

D6. Connected components in an undirected graph (DFS)

Given n vertices and edges, output the number of connected components and the vertices in each component.

Example input	Example output
n=5, edges=[(0,1),(1,2),(3,4)]	2 components: {0,1,2}, {3,4}
n=4, edges=[]	4 components: {0}, {1}, {2}, {3}
n=0, edges=[]	0 components (empty/invalid graph)

BFS problems

B1. Multi-source shortest distance / nearest facility (BFS)

Given a graph (or grid) and a set of sources S, compute the shortest distance from each node to its nearest source. Use multi-source BFS (all sources start with distance 0).

Example input	Example output
n=6, edges=[(0,1),(1,2),(2,3),(3,4),(4,5)], S={0,5}	dist=[0,1,2,2,1,0]
n=5, edges=[(0,1),(0,2),(0,3),(0,4)], S={2}	dist=[1,2,0,2,2]
n=4, edges=[(0,1)], S={0}	dist=[0,1,-1,-1] (-1 unreachable)

B2. Shortest escape in a building with elevators (BFS)

Floors 1..F. Start S, goal G. Buttons: up U floors or down D floors (within bounds). Find minimum button presses or report IMPOSSIBLE.

Example input	Example output
F=10, S=1, G=10, U=2, D=1	6
F=10, S=5, G=2, U=3, D=1	3
F=10, S=1, G=10, U=2, D=2	IMPOSSIBLE

B3. Minimum perfect squares sum (BFS)

Given n, find the minimum count of perfect squares whose sum is n (e.g., $12=4+4+4 \rightarrow 3$). BFS on remainders.

Example input	Example output
n = 12	3
n = 13	2 (9+4)
n = -1	0 (invalid n)

B4. Knight's minimum moves on a chessboard (BFS)

Given an $N \times N$ board, a start cell and a target cell, find the minimum number of knight moves (8 possible moves).

Example input	Example output
$N=8$, start=(0,0), target=(7,7)	6
$N=5$, start=(0,0), target=(4,4)	4
$N=2$, start=(0,0), target=(1,1)	-1 (unreachable)

B5. Word Ladder (BFS)

Given beginWord, endWord and a dictionary, change one letter at a time (each intermediate word must be in the dictionary). Find the minimum number of transformations.

Example input	Example output
begin="hit", end="cog", dict={hot,dot,dog,lot,log,cog}	4
begin="a", end="c", dict={a,b,c}	2
begin="hit", end="cog", dict={hot,dot,dog,lot,log}	0 (no solution)